SocketGrid: Sistema Distribuido Master–Worker con Latidos UDP, Control TCP y Prueba de Carga Sincronizada

Josue S. Vele Juan D. Figueroa
Trabajo en clase — Repositorio: github.com/JosueVele77/SocketGrid-Master
Octubre de 2026

***Resumen*—Este trabajo presenta SocketGrid, un sistema distribuido en Python formado por un coordinador (Master) y varios nodos trabajadores (Workers). La comunicación combina UDP, para latidos de bajo costo con confirmación (ACK) y detección de fallos por tiempo de espera, y TCP, para el registro, el envío de órdenes y la recepción fiable de informes. El sistema usa hilos y candados para atender tareas concurrentes, ejecuta tareas remotas (escaneo de puertos y consulta de registros) y coordina una prueba de carga con inicio sincronizado. Se describen la arquitectura, el protocolo y las decisiones de diseño, y se analizan los registros de una ejecución con dos Workers, en la que se verificó el registro en dos fases, el envío de órdenes y la exclusión mutua de tareas.**

***Términos de índice*—sockets, TCP, UDP, latidos, detección de fallos, concurrencia, sistemas distribuidos, Python.**

I. INTRODUCCIÓN

Los sistemas distribuidos requieren que varios procesos se coordinen a través de la red, detecten fallos y repartan trabajo. Estas necesidades se resuelven con protocolos de transporte distintos según el caso: UDP [1] es ligero y sin conexión, adecuado para señales periódicas, mientras que TCP [2] ofrece un flujo de bytes fiable y ordenado, adecuado para órdenes e informes.

SocketGrid combina ambos en una arquitectura Master–Worker. Sus contribuciones son: (a) un registro en dos fases (UDP y luego TCP) que evita sesiones de nodos desconocidos; (b) un detector de fallos basado en latidos con umbral de 6 s; (c) exclusión mutua de tareas por Worker, reflejada en el estado *OCUPADO*; y (d) una prueba de carga UDP con inicio sincronizado y límites de seguridad. El código fuente está disponible en el repositorio del proyecto.

El artículo se organiza así: la Sección II describe la arquitectura; la III, el protocolo; la IV, la implementación; la V, los resultados experimentales; la VI, el análisis de modificaciones y limitaciones; y la VII, las conclusiones.

II. ARQUITECTURA DEL SISTEMA

El Master escucha en tres puertos y ejecuta cuatro hilos de servicio más la consola interactiva en el hilo principal (Fig. 1). Cada Worker mantiene un socket UDP para latidos y una conexión TCP persistente con el Master; además lanza un hilo por tarea recibida.

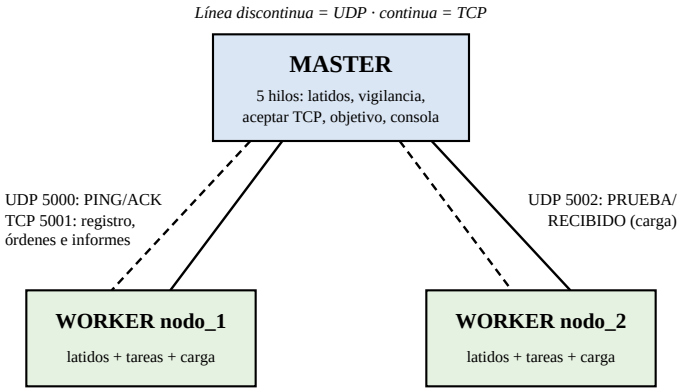


Fig. 1. Arquitectura Master–Worker y canales de comunicación.

TABLA I
PUERTOS Y FUNCIÓN

Puerto	Proto.	Función
5000	UDP	Latidos `PING` y confirmación `ACK`
5001	TCP	Registro, órdenes e informes
5002	UDP	Objetivo de la prueba de carga

A. Master

Los hilos de servicio son: recepción de latidos, vigilancia de nodos, aceptación de conexiones TCP y objetivo de carga. Además se crea un hilo por cada sesión TCP. El estado compartido consta de dos tablas, `nodos` (último latido, CPU y estado) y `conexiones` (socket TCP por nodo), protegidas por un candado.

B. Worker

El Worker valida su identificador (1 a 30 caracteres alfanuméricos, guion o guion bajo), se registra, y entonces inicia un hilo de latidos cada 2 s. Las órdenes se leen línea a línea; `FETCH_LOGS` se atiende en el hilo principal y las demás en un hilo propio, de modo que la recepción de órdenes nunca se bloquea.

III. PROTOCOLO DE COMUNICACIÓN

TCP entrega un flujo de bytes sin límites de mensaje, por lo que cada mensaje termina en un salto de línea y se lee con `makefile()` y `readline()`. Los campos se separan con `|`; por ello el identificador no admite `:` ni `|`.

TABLA II
MENSAJES DEL PROTOCOLO

Mensaje	Sentido	Descripción
`PING:id|cpu|estado`	W → M (UDP)	Latido; estado `ACTIVO` u `OCUPADO`
`ACK:id`	M → W (UDP)	Confirma el latido
`REGISTRO:id`	W → M (TCP)	Solicita sesión
`REGISTRADO:id`	M → W (TCP)	Acepta; si no, `ERROR:...`
`EXEC_TASK:scan_ports`	M → W (TCP)	Escanea puertos 5001–5003 del Master
`FETCH_LOGS`	M → W (TCP)	Devuelve los últimos 10 resultados
`BROADCAST_STRESS:ip:s|t0`	M → W (TCP)	Carga durante s segundos desde t0
`RESULTADO:id|...`	W → M (TCP)	Informe de la tarea
`PRUEBA / RECIBIDO`	W ↔ M (UDP 5002)	Petición y respuesta de carga

A. Registro en dos fases

El Worker envía un latido y exige un ACK válido proveniente del Master. Solo entonces abre la conexión TCP y envía `REGISTRO`. El Master rechaza la sesión si el identificador no existe en `nodos`, ya tiene una conexión o está marcado como caído.

B. Detección de fallos

Con un período de latido T = 2 s, el Master declara *CAÍDO* a un nodo tras 6 s sin latidos ($3T$), revisando cada 0.5 s. Este esquema corresponde a un detector de fallos por tiempo de espera [3]: un umbral bajo genera falsos positivos y uno alto retrasa la detección.

C. Prueba de carga sincronizada

El Master fija t_0 = *ahora* + 2 s y lo envía a todos los Workers activos. Cada Worker acepta la orden solo si el objetivo es la IP del Master, la duración está entre 1 y 10 s y t_0 está en [*ahora*−1, *ahora*+10]. Después espera hasta t_0 y lanza 4 hilos que envían `PRUEBA` con una pausa de 20 ms (hasta unas 50 peticiones/s por hilo), midiendo la latencia hasta `RECIBIDO`.

IV. IMPLEMENTACIÓN

Se empleó únicamente la biblioteca estándar de Python [4], [5]; no hay dependencias externas.

TABLA III
BIBLIOTECAS E INFLUENCIA

Módulo	Uso en el sistema
`socket`	Canales UDP/TCP: `bind, listen, accept, connect, sendto, recvfrom, sendall`
`threading`	Hilos, `Lock` (exclusión mutua) y `Event` (apagado)
`time`	`monotonic` (plazos), `time` (inicio común), `process_time` (CPU)
`sys`	Argumentos: IP del Master e identificador
`os`	`cpu_count` para el porcentaje de CPU

A. Sincronización

Se usan cuatro mecanismos: `candado` protege las tablas del Master; `candado_consola` evita salidas mezcladas; `candado_envio` serializa las escrituras al socket TCP del Worker; y `candado_tarea`, adquirido sin bloqueo, permite una sola tarea pesada por Worker. Si el candado está tomado, el Worker informa `Worker ocupado` y el latido indica `OCUPADO`. Cada hilo de carga acumula en su propia lista (recibidos, errores, latencia), lo que evita condiciones de carrera al sumar.

B. Apagado ordenado

Una bandera `detener` y *timeouts* de 0.5 s en los sockets permiten que todos los hilos terminen. Los bloques `finally` liberan candados, cierran sockets y eliminan la conexión de la tabla solo si corresponde a la sesión actual.

C. Uso de CPU

El Worker estima su carga como `100·Δprocess_time / (Δmonotonic·núcleos)`, es decir, el porcentaje de la capacidad del equipo que consume el proceso.

V. PRUEBAS Y RESULTADOS

Entorno. Windows PowerShell dentro de PyCharm, en una misma máquina (127.0.0.1, *loopback*): un Master (`python master.py`) y dos Workers (`python worker.py nodo_1` y `nodo_2`) en terminales independientes. Las Figs. 2–4 muestran los registros obtenidos.

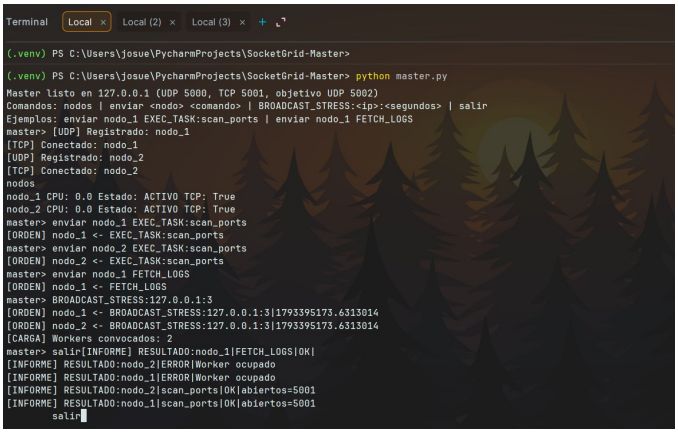


Fig. 2. Consola del Master: registro UDP/TCP de los nodos, comando *nodos*, órdenes enviadas e informes recibidos.

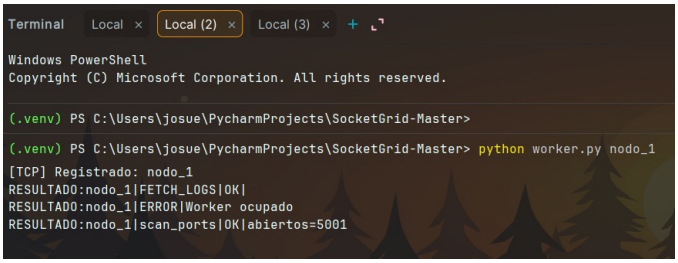


Fig. 3. Salida del Worker nodo_1.

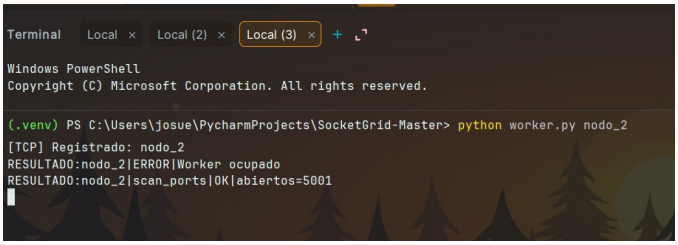


Fig. 4. Salida del Worker nodo_2.

TABLA IV
EVENTOS OBSERVADOS

Evento	Resultado observado
Registro de nodos	Master: `[UDP] Registrado` y `[TCP] Conectado` para nodo_1 y nodo_2
Comando `nodos`	Ambos `ACTIVO`, CPU 0.0, TCP `True`
`scan_ports` en ambos	`abiertos=5001` en nodo_1 y nodo_2
`FETCH_LOGS` en nodo_1	`RESULTADO:nodo_1|FETCH_LOGS|OK|` (detalle vacío)
`BROADCAST_STRESS:127.0.0.1:3`	Enviado a 2 Workers (`t0=1793395173.63`); ambos respondieron `ERROR|Worker ocupado`

A. Análisis

- **Registro y estado.** Los dos nodos completaron las dos fases del registro y el comando `nodos` los reportó activos con sesión TCP, lo que confirma la coordinación entre los canales UDP y TCP.
- **Escaneo.** Solo se detectó el puerto 5001. Es lo esperado: el escaneo usa `connect_ex` (TCP), por lo que el puerto 5002 (UDP) no aparece como abierto y el 5003 no tiene servicio.
- **Exclusión mutua.** La orden de carga llegó mientras ambos Workers aún ejecutaban `scan_ports`. Como una tarea retiene `candado_tarea`, ambos rechazaron la carga con `Worker ocupado`; en nodo_1 este error aparece antes del resultado del escaneo, lo que es consistente con esa secuencia. Es el comportamiento de diseño, pero implica que **en esta ejecución no se obtuvieron métricas de la prueba de carga** (peticiones por segundo ni latencia).
- **Registros.** `FETCH_LOGS` devolvió detalle vacío porque aún no había resultados guardados; además, las consultas no se almacenan para no repetirse en el historial.

VI. MODIFICACIONES, LIMITACIONES Y TRABAJO FUTURO

La Tabla V resume los parámetros más sensibles; la guía de estudio del repositorio detalla más casos.

TABLA V
PARÁMETROS CRÍTICOS

Cambio	Efecto esperado
Umbral de caída 6 s → 2 s	Falsos positivos por retrasos normales
Latido 2 s → 10 s	Workers sanos marcados como caídos
Quitar `\n` al enviar	`readline()` espera hasta agotar el *timeout*
Quitar `candado_envio`	Mensajes TCP entrelazados
Hilos de carga 4 → 100	Mayor carga; pérdida de paquetes y errores
Pausa 20 ms → 0	Saturación de CPU y del socket del Master
Cambiar un puerto en un solo archivo	El Worker no encuentra al Master
Quitar la validación del objetivo	El Worker podría enviar tráfico a terceros

Limitaciones. UDP no garantiza entrega, por lo que un latido perdido puede retrasar el estado; no hay autenticación ni cifrado; el Worker no se reconecta automáticamente; el escaneo cubre solo TCP; y las pruebas se hicieron en *loopback*, sin latencia ni pérdidas de una red real.

Trabajo futuro. Repetir la prueba de carga una vez finalizada la tarea previa para obtener métricas; reconexión automática; autenticación con clave compartida y TLS; mensajes en JSON; y configuración mediante argumentos en lugar de constantes.

VII. CONCLUSIONES

SocketGrid demuestra que la combinación de UDP para señales periódicas y TCP para control fiable da lugar a un sistema simple y robusto. El registro en dos fases, el detector por tiempo de espera y la exclusión mutua de tareas funcionaron como se diseñaron en la ejecución documentada. El rechazo de la carga por Worker ocupado evidencia que el control de concurrencia es efectivo y que las órdenes pesadas deben secuenciarse.

REFERENCIAS

[1] J. Postel, “User Datagram Protocol,” RFC 768, IETF, ago. 1980.

[2] W. Eddy, Ed., “Transmission Control Protocol (TCP),” RFC 9293, IETF, ago. 2022.

[3] T. D. Chandra and S. Toueg, “Unreliable failure detectors for reliable distributed systems,” *J. ACM*, vol. 43, no. 2, pp. 225–267, 1996.

[4] Python Software Foundation, “socket — Low-level networking interface,” *The Python Standard Library*. [En línea]. Disponible: https://docs.python.org/3/library/socket.html

[5] Python Software Foundation, “threading — Thread-based parallelism,” *The Python Standard Library*. [En línea]. Disponible: https://docs.python.org/3/library/threading.html

[6] A. S. Tanenbaum and M. van Steen, *Distributed Systems: Principles and Paradigms*, 2nd ed. Upper Saddle River, NJ, USA: Pearson, 2007.

[7] J. S. Vele and J. D. Figueroa, “SocketGrid-Master,” repositorio de GitHub, 2026. [En línea]. Disponible: https://github.com/JosueVele77/SocketGrid-Master